Model subiect test practic APD

Să se implementeze, **folosind Pthreads**, un program paralel pentru sortarea unui vector cu N elemente prin intermediul Rank Sort, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor.

Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească fragmentele parțiale de vector sortate de celelalte thread-uri.

În urma compilării, va rezulta un executabil numit *testapd*, care va primi ca parametri valorile lui N și P (în această ordine). Vectorul de sortat va fi inițializat în funcția *main* ca `v[i] = N - i + 5`, înainte de pornirea thread-urilor. Aveți mai jos trei exemple de rulare:

```
$ ./testapd 10 3
6 7 8 9 10 11 12 13 14 15

$ ./testapd 25 4
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

$ ./testapd 3 8
6 7 8
```

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât vectorul sortat. Afișarea vectorului sortat se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N și P sunt: $0 < N <= 10000$, $0 < P <= 8$. Programul trebuie să funcționeze corect pentru orice combinație de N și P din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă, un Makefile cu target-ul *build* și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.

Model subiect test practic APD

Să se implementeze, **folosind Java Threads**, un program paralel pentru sortarea unui vector cu N elemente prin intermediul Rank Sort, folosind P thread-uri. Rezolvarea se va face maximizând concurența thread-urilor.

Rezultatul se va printa de către un thread ales de către programator (poate fi thread-ul cu un anumit ID), dar **NU** de către thread-ul *main*. Mai mult, doar acestui thread îi va fi permis să citească fragmentele parțiale de vector sortate de celelalte thread-uri.

În urma compilării prin intermediul comenzii `javac *.java`, va rezulta un fișier numit *Testapd.class*, care va primi ca parametri valorile lui N și P (în această ordine). Vectorul de sortat va fi inițializat în funcția *main* ca `v[i] = N - i + 5`, înainte de pornirea thread-urilor. Aveți mai jos trei exemple de rulare:

```
$ javac *.java
[...]

$ java Testapd 10 3
6 7 8 9 10 11 12 13 14 15

$ java Testapd 25 4
6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30

$ java Testapd 3 8
6 7 8
```

Atenție! Toate thread-urile trebuie pornite în același timp (într-o buclă comună). Programul submis nu trebuie să printeze altceva decât vectorul sortat. Afișarea vectorului sortat se va face dintr-unul din thread-urile pornite de către voi.

Se presupune că parametrii de intrare sunt corecți (atât ca număr și ordine, cât și ca valori). Valorile acceptate pentru N și P sunt: $0 < N <= 10000$, $0 < P <= 8$. Programul trebuie să funcționeze corect pentru orice combinație de N și P din aceste intervale.

Trebuie să încărcați pe Moodle o arhivă care să conțină **în rădăcină** fișierele sursă și eventual un fișier README.

Atenție! Fișierul README este necesar doar în cazul în care considerați că este nevoie de explicații adiționale în legătura cu implementarea realizată.